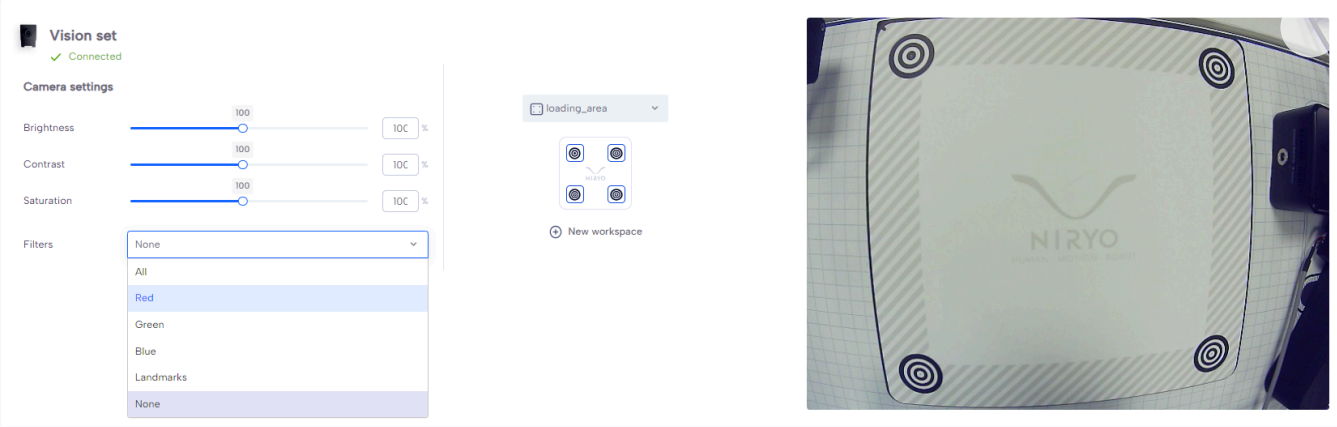
Use the Vision Set

This document shows how to use Ned's Vision Set

With NiryoStudio

You can acess the vision Set on NiryoStudio in the Equipment tab. First you will need to scan the tools to get the vision section available.



From here you can:

- Monitor the video Stream.
- Change camera settings like the Brightness , the Contrast or the Saturation.
- Use filter to detect different colors or the landmarks
- Manage your vision workspaces and check if it is correctly set by clicking its landmarks.
  The robot will move to the landmark position when clicking on it, so be certain the trajectory is collision free

Workspaces

The Vision Set uses a 2D camera to detect objects and calculate their positions in space. This detection allows you to trigger the actions of your choice.

To allow such functions with a 2D camera, we developed a system based on landmarks that define workspaces in 3 dimensions (the third dimension being the height, taking the height of the landmarks as origin).



For each action related to Vision, the camera will give you a relative position in the workspace reference that the robot will translate to its own reference.

> Please note that any action related to Vision will need to be done in a workspace, which means all 4 markers should be detectable by the camera

> The first landmark is slightly different from the others : it has only 3 blacks areas, which allows to sort the landmarks clockwise.

Conditions

To use the Vision related functions, two conditions are required:

- Workspace's landmarks positions are set and mechanically fixed
- The landmarks delimiting the workspace are visible

> Please note that these two conditions need to be met to allow the use of buil-in vision functions.

With Blockly

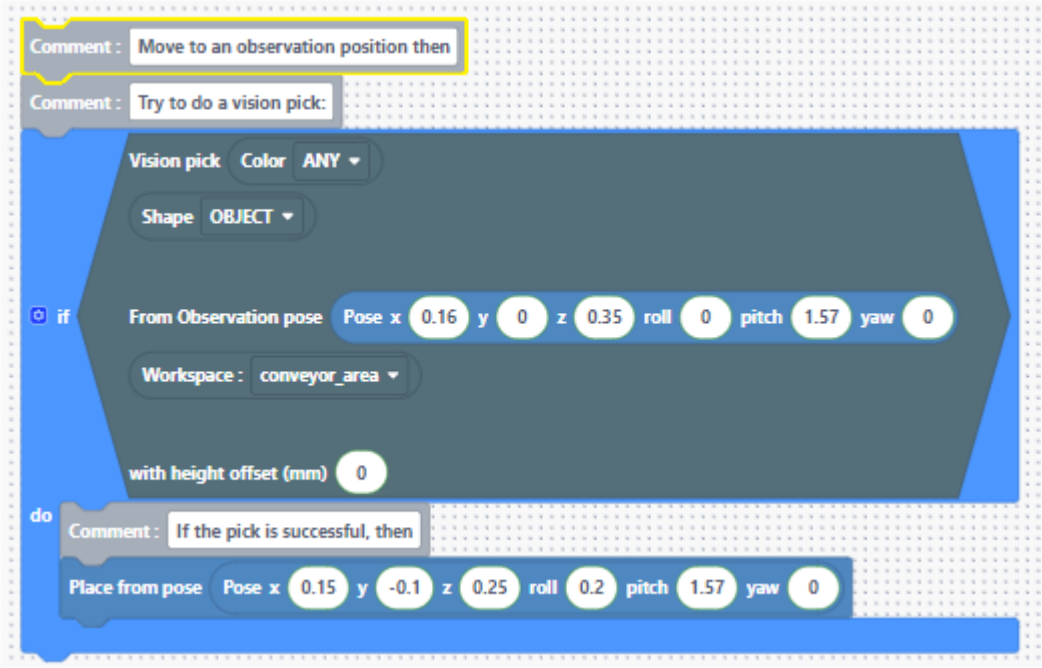
Simple Vision pick and place

Task

Ned has to go to an observation pose above the workspace and pick an object from its workspace using vision. Then, it should place it anywhere.

Solution & explanation

To pick an object using Vision, you can use the **Vision Pick** Block. It will return `True` if an object is detected and picked, so it **has** to be put in a logic condition block ⇨ if the action happens, we can place the picked object. You can find th example in the Templates of the vision tab of Blockly.



Empty the Workspace with Vision pick and place

Task 1

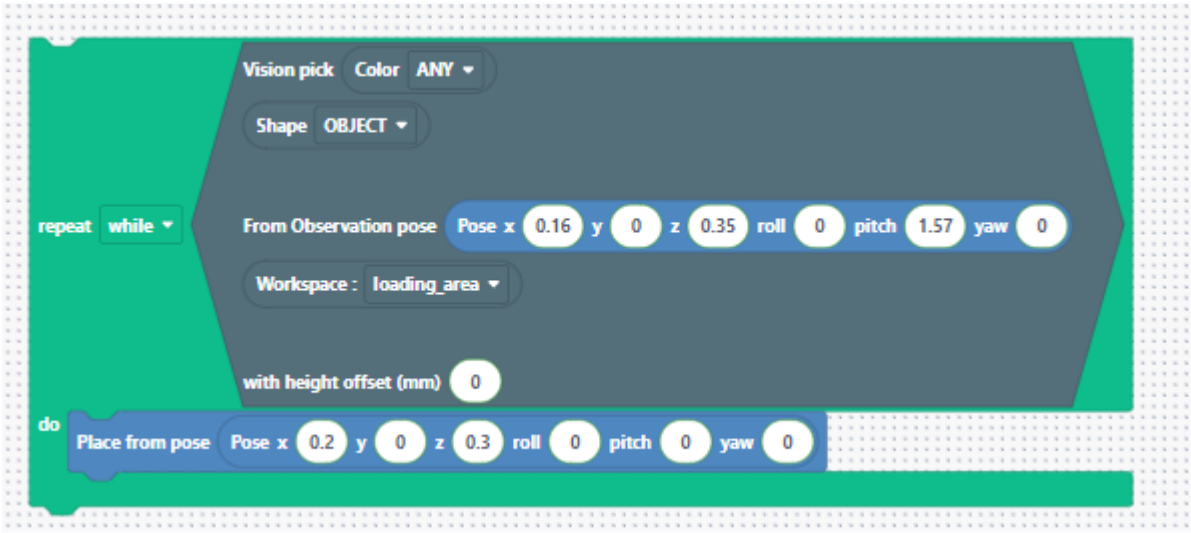
Ned has to pick all the objects in the workspace one by one then place them all on the same spot.

The workspace can have any number of object without fixed color/shape.

Solution & explanation 1

To make the Vision pick task, you can use the Vision Pick with Observation Pose Block which will return `True` each time the action happens. By using a loop as a condition (here,

`repeat while`), the robot will continue the process until it picked all the objects of the workspace.



In this sequence, Ned will pick all the objects it can detect on the workspace then activate its learning mode.

Multi-reference packaging with Vision pick & place

Task 2

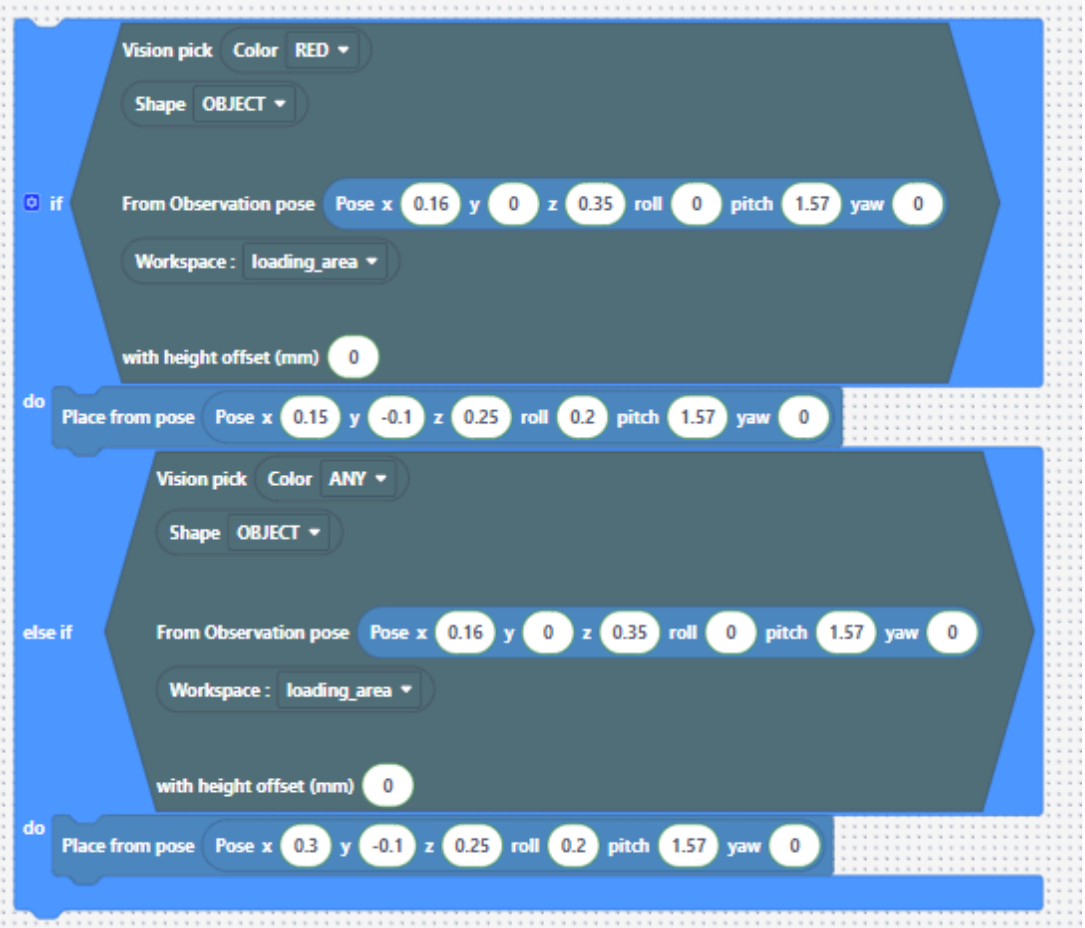
Ned has to take all the objects from the workspace and place them at different positions depending on their attributes ⇨ the red objects will be placed in an area and all the other ones in a different area.

Solution & explanation 2

You will start the process by using the Vision Pick with Observation Pose Block to interact with the red objects only. If the action happens, the object will be placed on a certain pose.

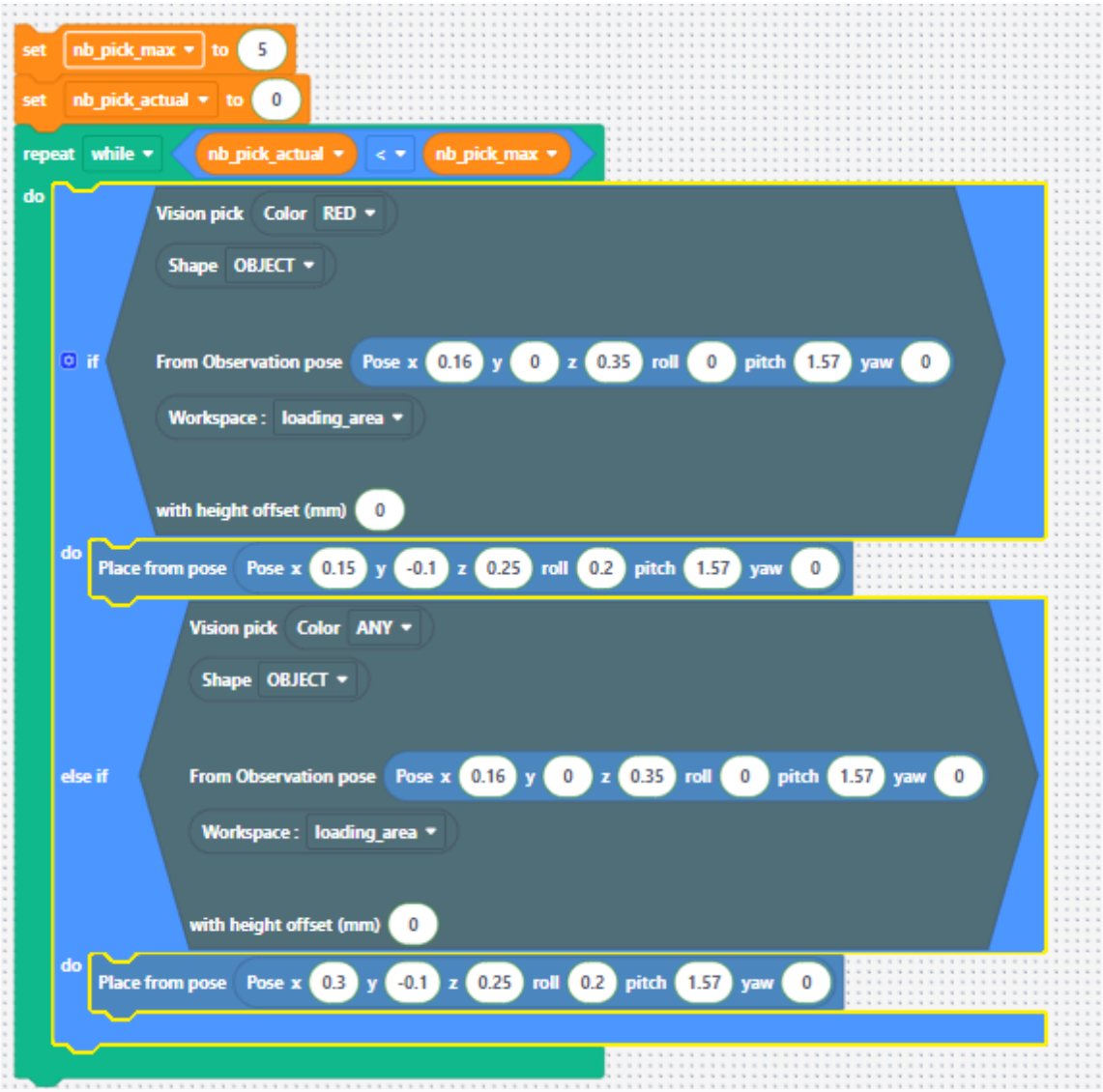


Then, a `else if` condition is added. That will be triggered if the first vision pick did not happen, which means that there are no red objects anymore.



With this program, Ned2 will try to pick a red object, then place it in an area. If there is no red object in the workspace, Ned2 will take another object to place it in another area.

Now, we can use a loop to repeat this sequence for a given amount of objects using variables.



With PyNiryo

> In order to achieve the following examples, you need to create a vision workspace. In this page, the workspace used is named `workspace_1`. To create it, the user should go on NiryoStudio.

> All the following examples are only a part of what can be made with PyNiryo in terms of Vision. We advise you to look at PyNiryo API to understand more deeply

Simple Vision Pick & Place

The goal of a Vision Pick & Place is the same as a classical Pick & Place, with a close difference: the camera detects where the robot has to go in order to pick!

This short example shows how to do your first Vision pick using the **`vision_pick()`** function:

python

```python
# Imports
from pyniryo import *

# - Constants
workspace_name = "workspace_1"  # Robot's Workspace Name
robot_ip_address = "x.x.x.x"

# The pose from where the image processing happens
observation_pose = PoseObject(
    x=0.16, y=0.0, z=0.35,
    roll=0.0, pitch=1.57, yaw=0.0,
)
# Place pose
place_pose = PoseObject(
    x=0.0, y=-0.2, z=0.12,
    roll=0.0, pitch=1.57, yaw=-1.57
)

# - Initialization

# Connect to robot
robot = NiryoRobot(robot_ip_address)
# Calibrate robot if the robot needs calibration
robot.calibrate_auto()
# Updating tool
robot.update_tool()

# --- -------------- --- #
# --- CODE GOES HERE --- #
# --- -------------- --- #
robot.move_pose(observation_pose)
# Trying to pick target using camera
obj_found, shape_ret, color_ret = robot.vision_pick(workspace_name)
if obj_found:
    robot.place_from_pose(place_pose)


robot.close_connection()
```

Code Details - Simple Vision Pick and Place

To execute a Vision pick, we firstly need to go to a place where the robot will be able to see the workspace:

python

```python
robot.move_pose(observation_pose)
```

Then, we try to perform a Vision pick in the workspace with the `vision_pick()` function:

python

```
obj_found, shape_ret, color_ret = robot.vision_pick(workspace_name)
```

Variables `shape_ret` and `color_ret` are respectively of type `ObjectShape` and `ObjectColor`, and store the shape and the color of the detected object! We will not use them for this first example.

The `obj_found` variable is a boolean which indicates whereas an object has been found and picked, or not. Thus, if the pick worked, we can place the object at the place pose.

python

```
if obj_found:
    robot.place_from_pose(place_pose)
```

Finally, we close the connection with the robot:

python

```
robot.close_connection()
```

> If your `obj_found` variable indicates `False`, check that:
> - Nothing obstructs the camera field of view
> - Workspace's 4 markers are visible
> - At least 1 object is placed fully inside the workspace

First conditioning via Vision

In most of use cases, the robot will need to perform more than one Pick & Place. In this example, we will see how to condition multiple objects according to a straight line:

python

```
# Imports
from pyniryo import *

# - Constants
workspace_name = "workspace_1"  # Robot's Workspace Name
robot_ip_address = "x"

# The pose from where the image processing happens
observation_pose = PoseObject(
    x=0.16, y=0.0, z=0.35,
```

```
    roll=0.0, pitch=1.57, yaw=0.0,
)
# Place pose
place_pose = PoseObject(
    x=0.0, y=-0.2, z=0.12,
    roll=0.0, pitch=1.57, yaw=-1.57
)

# - Initialization

# Connect to robot
robot = NiryoRobot(robot_ip_address)
# Calibrate robot if the robot needs calibration
robot.calibrate_auto()
# Updating tool
robot.update_tool()

# --- -------------- --- #
# --- CODE GOES HERE --- #
# --- -------------- --- #
# Initializing variables
offset_size = 0.05
max_catch_count = 4

# Loop until enough objects have been caught
catch_count = 0
while catch_count < max_catch_count:
    # Moving to observation pose
    robot.move_pose(observation_pose)

    # Trying to get object via Vision Pick
    obj_found, shape, color = robot.vision_pick(workspace_name)
    if not obj_found:
        robot.wait(0.1)
        continue

    # Calculate place pose and going to place the object
    next_place_pose = place_pose.copy_with_offsets(x_offset=catch_count * offset_s
    robot.place_from_pose(next_place_pose)

    catch_count += 1

robot.go_to_sleep()

robot.close connection()
```

Code Details - First Conditioning via Vision

We want to catch `max_catch_count` objects, and space each of them by `offset_size` meter:

python

```
offset_size = 0.05
max_catch_count = 4
```

We start a loop until the robot has caught `max_catch_count` objects:

python

```python
catch_count = 0
while catch_count < max_catch_count:
```

For each iteration, we firstly go to the observation pose and then, try to make a Vision pick in the workspace:

python

```python
robot.move_pose(observation_pose)

obj_found, shape, color = robot.vision_pick(workspace_name)
```

If the Vision pick failed, we wait 0.1 second and then, start a new iteration:

python

```python
if not obj_found:
    robot.wait(0.1)
    continue
```

Else, we compute the new place position according to the number of catches, and then, go placing the object at that place:

python

```python
next_place_pose = place_pose.copy_with_offsets(x_offset=catch_count * offset_size)
robot.place_from_pose(next_place_pose)
```

We also increment the `catch_count` variable:

python

```python
catch_count += 1
```

Once the target catch number is achieved, we go to sleep: That will make the robot go to home position then activate the le

python

```python
robot.go_to_sleep()
```

Multi Reference Conditioning

During a conditioning task, objects may not always be placed as the same place according to their type. In this example, we will see how to align object according to their color, using the color element **`ObjectColor`** returned by **`vision_pick()`** function:

python

```python
# Imports
from pyniryo import *

# - Constants
workspace_name = "workspace_1"  # Robot's Workspace Name
robot_ip_address = "x.x.x.x"

# The pose from where the image processing happens
observation_pose = PoseObject(
    x=0.16, y=0.0, z=0.35,
    roll=0.0, pitch=1.57, yaw=0.0,
)
# Place pose
place_pose = PoseObject(
    x=0.0, y=-0.2, z=0.12,
    roll=0.0, pitch=1.57, yaw=-1.57
)

# - Initialization

# Connect to robot
robot = NiryoRobot(robot_ip_address)
# Calibrate robot if the robot needs calibration
robot.calibrate_auto()
# Updating tool
robot.update_tool()

# --- -------------- --- #
# --- CODE GOES HERE --- #
# --- -------------- --- #
# Distance between elements
offset_size = 0.05
max_failure_count = 3

# Dict to write catch history
count_dict = {
    ObjectColor.BLUE: 0,
    ObjectColor.RED: 0,
    ObjectColor.GREEN: 0,
}

try_without_success = 0
# Loop until too much failures
while try_without_success < max_failure_count:
    # Moving to observation pose
    robot.move_pose(observation_pose)
    # Trying to get object via Vision Pick
    obj_found, shape, color = robot.vision_pick(workspace_name)
    if not obj_found:
        try_without_success += 1
```

```
            robot.wait(0.1)
            continue

        # Choose X position according to how the color line is filled
        offset_x_ind = count_dict[color]

        # Choose Y position according to ObjectColor
        if color == ObjectColor.BLUE:
            offset_y_ind = -1
        elif color == ObjectColor.RED:
            offset_y_ind = 0
        else:
            offset_y_ind = 1

        # Going to place the object
        next_place_pose = place_pose.copy_with_offsets(x_offset=offset_x_ind * offset_
                                                       y_offset=offset_y_ind * offset_
        robot.place_from_pose(next_place_pose)

        # Increment count
        count_dict[color] += 1
        try_without_success = 0

robot.go_to_sleep()
robot.close_connection()
```

Code Details - Multi Reference Conditioning

We want to catch objects until Vision Pick failed `max_failure_count` times. Each of the object will be put on a specific column according to its color. The number of catches for each color will be stored on a dictionary `count_dict` .

python

```
# Distance between elements
offset_size = 0.05
max_failure_count = 3

# Dict to write catch history
count_dict = {
    ObjectColor.BLUE: 0,
    ObjectColor.RED: 0,
    ObjectColor.GREEN: 0,
}

try_without_success = 0
# Loop until too much failures
while try_without_success < max_failure_count:
```

For each iteration, we firstly go to the observation pose and then, try to make a Vision pick in the workspace:

python

```python
robot.move_pose(observation_pose)

obj_found, shape, color = robot.vision_pick(workspace_name)
```

If the Vision pick failed, we wait 0.1 second and then, start a new iteration, without forgetting to increment the failure counter:

python

```python
if not obj_found:
    try_without_success += 1
    robot.wait(0.1)
    continue
```

Else, we compute the new place position according to the number of catches, and then, go place the object at that place:

python

```python
# Choose X position according to how the color line is filled
offset_x_ind = count_dict[color]

# Choose Y position according to ObjectColor
if color == ObjectColor.BLUE:
    offset_y_ind = -1
elif color == ObjectColor.RED:
    offset_y_ind = 0
else:
    offset_y_ind = 1

# Going to place the object
next_place_pose = place_pose.copy_with_offsets(x_offset=offset_x_ind * offset_size
                                               y_offset=offset_y_ind * offset_size
robot.place_from_pose(next_place_pose)
```

We increment the `count_dict` dictionary and reset `try_without_success` :

python

```python
count_dict[color] += 1
try_without_success = 0
```

Once the target catch number is achieved, we go to sleep:

python

```python
robot.go_to_sleep()
```

Sorting Pick with Conveyor

An interesting way to bring objects to the robot, is the use of a Conveyor Belt. In this examples, we will see how to catch only a certain type of object by stopping the conveyor as soon as the object is detected on the workspace.

python

```python
# Imports
from pyniryo import *

# - Constants
workspace_name = "workspace_1"  # Robot's Workspace Name
robot_ip_address = "x.x.x.x"

# The pose from where the image processing happens
observation_pose = PoseObject(
    x=0.154, y=0.058, z=0.284,
    roll=2.12, pitch=1.523, yaw=2.14,
)

# Place pose
place_pose = PoseObject(
    x=0.0, y=-0.2, z=0.12,
    roll=0.0, pitch=1.57, yaw=-1.57
)

# - Initialization

# Connect to robot
robot = NiryoRobot(robot_ip_address)
# Calibrate robot if the robot needs calibration
robot.calibrate_auto()
# Updating tool
robot.update_tool()

# --- -------------- --- #
# --- CODE GOES HERE --- #
# --- -------------- --- #
# Initializing variables
offset_size = 0.05
max_catch_count = 4
shape_expected = ObjectShape.CIRCLE
color_expected = ObjectColor.

conveyor_id = robot.set_conveyor()

catch_count = 0
while catch_count < max_catch_count:
    # Turning conveyor on
    robot.run_conveyor(conveyor_id)
    # Moving to observation pose
    robot.move_pose(observation_pose)
    # Check if object is in the workspace
    obj_found, pos_array, shape, color = robot.detect_object(workspace_name,
                                                             shape=shape_expected,
```

```
                                                                color=color_expected)
    if not obj_found:
        robot.wait(0.5)  # Wait to let the conveyor turn a bit
        print(f"waiting for {color_expected.name} {shape_expected.name} ...")
        continue
    # Stopping conveyor
    robot.stop_conveyor(conveyor_id)
    # Making a vision pick
    obj_found, shape, color = robot.vision_pick(workspace_name,
                                                shape=shape_expected,
                                                color=color_expected)
    if not obj_found:  # If visual pick did not work
        continue

    # Calculate place pose and going to place the object
    next_place_pose = place_pose.copy_with_offsets(x_offset=catch_count * offset_s
    robot.place_from_pose(next_place_pose)

    catch_count += 1

# Stopping & unsetting conveyor
robot.stop_conveyor(conveyor_id)
robot.unset_conveyor(conveyor_id)

robot.go_to_sleep()
robot.close_connection()
```

Code Details - Sort Picking

Firstly, we initialize your process: we want the robot to catch 4 red circles. To do so, we set variables `shape_expected` and `color_expected` with **`ObjectShape.CIRCLE`** and **`ObjectColor.RED`** .

python

```
offset_size = 0.05
max_catch_count = 4
shape_expected = ObjectShape.CIRCLE
color_expected = ObjectColor.RED
```

We activate the connection with the Conveyor Belt and start a loop until the robot has caught `max_catch_count` objects:

python

```
conveyor_id = robot.set_conveyor()

catch_count = 0
while catch_count < max_catch_count:
```

For each iteration, we firstly run the Conveyor Belt (if the latter is already running, nothing will happen), then go to the observation pose:

python

```python
# Turning the Conveyor Belt on
robot.run_conveyor(conveyor_id)
# Moving to observation pose
robot.move_pose(observation_pose)
```

We then check if an object corresponding to our criteria is in the workspace. If not, we wait 0.5 second and then, start a new iteration:

python

```python
obj_found, pos_array, shape, color = robot.detect_object(workspace_name,
                                                         shape=shape_expected,
                                                         color=color_expected)
if not obj_found:
    robot.wait(0.5)  # Wait to let the conveyor turn a bit
    continue
```

Else, stop the Conveyor Belt and try to make a Vision pick:

python

```python
# Stopping Conveyor Belt
robot.stop_conveyor(conveyor_id)
# Making a Vision pick
obj_found, shape, color = robot.vision_pick(workspace_name,
                                            shape=shape_expected,
                                            color=color_expected)
if not obj_found:  # If visual pick did not work
    continue
```

If Vision Pick succeed, compute new place pose, and place the object:

python

```python
# Calculate place pose and going to place the object
next_place_pose = place_pose.copy_with_offsets(x_offset=catch_count * offset_size)
robot.place_from_pose(next_place_pose)

catch_count += 1
```

Once the target catch number is achieved, we stop the Conveyor Belt and go to sleep:

python

```python
# Stopping & unsetting Conveyor Belt
robot.stop_conveyor(conveyor_id)
```

```
robot.unset_conveyor(conveyor_id)

robot.go_to_sleep()
```

With Python ROS Wrapper

Simple Vision pick

> In order to achieve the following examples, you need to create a vision workspace. In this page, the workspace used is named `workspace_1`. To create it, the user should go on NiryoStudio.

python

```python
#!/usr/bin/env python

# Imports
from niryo_robot_python_ros_wrapper import *
import rospy

# Initializing ROS node
rospy.init_node('niryo_ned_example_python_ros_wrapper')

niryo_robot = NiryoRosWrapper()

# - Constants
workspace_name = "workspace_1"  # Robot's Workspace Name

# The observation pose
observation_pose = (0.18, 0., 0.35, 0., 1.57, -0.2)
# The Place pose
place_pose = (0., -0.25, 0.1, 0., 1.57, -1.57)

# - Main Program
# Calibrate robot if robot needs calibration
niryo_robot.calibrate_auto()
# Changing tool
niryo_robot.update_tool()

niryo_robot.move_pose(*observation_pose)
# Trying to pick target using camera
ret = niryo_robot.vision_pick(workspace_name,
                              height_offset=0.0,
                              shape=ObjectShape.ANY,
                              color=ObjectColor.ANY)
obj_found, shape_ret, color_ret = ret


```

```
    if obj_found:
        niryo_robot.place_from_pose(*place_pose)
```

With Modbus

```
from pymodbus.client import ModbusTcpClient
from pymodbus.payload import BinaryPayloadBuilder, BinaryPayloadDecoder
import time
from enum import Enum, unique

"""
This code is an example of how to use two available vision related
function in the Modbus TCP server from the client side.
"""
# Enums for shape and color. Those enums are the one used by the modbus server to rece
@unique
class ColorEnum(Enum):
    ANY = 0
    BLUE = 3
    RED = 1
    GREEN = 2
    NONE = -1

@unique
class ShapeEnum(Enum):
    ANY = 0
    CIRCLE = 1
    SQUARE = 2
    TRIANGLE = 3
    NONE = -1

# Functions to convert variables for/from registers
def encode(value, data_type):
    if not isinstance(value, list):
        value = [value]

    if data_type == int:
        # No need to encode an int
        return value

    encoder = BinaryPayloadBuilder()

    if data_type == float:
        for v in value:
            encoder.add_32bit_float(v)
    elif data_type == str:
        for v in value:
            encoder.add_string(v)
    else:
        raise TypeError(f'Unknown type "{data_type}"')
    return encoder.to_registers()

def decode(value: list, data_type):
```

```
    decoder = BinaryPayloadDecoder.fromRegisters(value)
    if data_type == int:
        r_value = value
    elif data_type == float:
        if len(value) % 2 != 0:
            raise ValueError("Can't decode floats with an uneven number of bytes")
        n_float = len(value) // 2
        r_value = [decoder.decode_32bit_float() for _ in range(n_float)]
    elif data_type == str:
        decoded_string = decoder.decode_string(200).decode()
        if decoded_string[-1] == '\x00':
            decoded_string = decoded_string[:-1]
        r_value = [decoded_string]
    else:
        print("Invalid data type. Supported types: int, float, str, bool")
        r_value = []
    return r_value if len(r_value) > 1 else r_value[0]


# ----------- Modbus server related function

def back_to_observation():
    joint_real = [-1.745, 0.568, -0.834, -0.092, -1.445, -0.118]
    joint_real_to_send = encode(joint_real,float)
    # store the joint values
    client.write_registers(50, joint_real_to_send)
    # set the move mode to Joint
    client.write_register(74, 0)
    # # collision detected
    if client.read_coils(117):
        client.write_coils(117, False)
    # send move command
    client.write_coil(113, 1)
    # Wait for end of Move command
    while client.read_holding_registers(113, count=1).registers[0] == 1:
        time.sleep(0.01)

def open_tool():
    client.write_coil(51, 0)


def close_tool():
    client.write_coil(51, 1)


def register_workspace_name(ws_name):
    workspace_request_register = encode(ws_name,str)
    client.write_registers(119, workspace_request_register)


def register_height_offset(height_offset):
    client.write_register(139, height_offset)


def auto_calibration():
    print("Calibrate Robot if needed ...")
    client.write_coil(116, 1)
```

```
def get_current_tool_id():
    return client.read_input_registers(121, count=1).registers[0]


# Functions to call Modbus Server vision functions

## STOPED HERE##
def detect_object(workspace_str, shape_int, color_int):
    register_workspace_name(workspace_str)

    # write requested values in registers
    client.write_registers(140, shape_int)
    client.write_registers(141, color_int)

    # Wait for end of detect_object function
    while client.read_holding_registers(52, count=1).registers[0] == 1:
        time.sleep(0.01)

    # - Check result : POSE
    obj_rel_pose = []
    result_shape_int = -1
    result_color_int = -1
    obj_found = client.read_discrete_inputs(52).bits[0]
    if obj_found:
        pose = decode(client.read_input_registers(86, count=12).registers,float)
        obj_rel_pose = pose
        # - Check result : SHAPE AND COLOR
        result_shape_int = client.read_input_registers(98).registers[0]
        result_color_int = client.read_input_registers(99).registers[0]

    return obj_rel_pose, result_shape_int, result_color_int



# ----------- Main programm

if __name__ == '__main__':
    print("--- START")

    client = ModbusTcpClient('192.168.1.207', port=5020)

    # -------- Variable definition
    workspace_name = 'loading_area'
    height_offset = 0
    intermediate_joint = [-1.746, 0.383, -0.978, -0.106, -0.822, -0.11]
    place_pose = [0.092, (-0.287), 0.086, 2.927, 1.504, 1.805]
    nb_square = 0
    nb_circle = 0

    # connect to modbus server
    client.connect()
    print("Connected to modbus server")

    # launch auto calibration then go to obs. pose
    auto_calibration()
    back_to_observation()
```

```
    # updatetool
    client.write_coil(50,1)

    # ------------------- DETECT, PICK AND PLACE ACCORDING ON THE SHAPE --------------

    shape = ShapeEnum.ANY.value
    color = ColorEnum.ANY.value

    while (detect_object(workspace_name, shape, color)[1] != ShapeEnum.NONE.value):
        # while objects are detected in the workspace, grab them and stack them
        # according to their shape (circle or square).

        obj_rel_pose, shape_found, color_found = detect_object(workspace_name, shape,
        print('Detected a', ColorEnum(color_found).name, ShapeEnum(shape_found).name)
        open_tool()
        pose_to_send = encode(obj_rel_pose,float)
        client.write_registers(62, pose_to_send)
        # set the move mode to Pose
        client.write_register(74, 1)
        # # collision detected
        if client.read_coils(117):
            client.write_coils(117, False)
        # send move command
        client.write_coil(113, 1)
        # Wait for end of Move command
        while client.read_holding_registers(113, count=1).registers[0] == 1:
            time.sleep(0.01)
        close_tool()


        # ---- Go to release pose
        pose_to_send = encode(place_pose,float)
        client.write_registers(62, pose_to_send)
        # set the move mode to Pose
        client.write_register(74, 1)
        # # collision detected
        if client.read_coils(117):
            client.write_coils(117, False)
        # send move command
        client.write_coil(113, 1)
        # Wait for end of Move command
        while client.read_holding_registers(113, count=1).registers[0] == 1:
            time.sleep(0.01)

        # ---- Release pawn and go back to observation
        open_tool()
        back_to_observation()

    print('No more objects detected in the workspace')

    # Activate learning mode and close connexion
    client.write_register(300, 1)
    client.close()
    print("Close connection to modbus server")
    print("--- END")
```

Go further

To go further , please refer to the API documentation

Made with Docsie.io